

## Geometrijski algoritmi

Geometrijski algoritmi igraju važnu ulogu u mnogim oblastima, na primer u računarskoj grafici, projektovanju pomoću računara, projektovanju VLSI (integriranih kola visoke rezolucije), robotici i bazama podataka. U računarski generisanoj slici može biti na hiljade ili čak milione tačaka, linija, kvadrata ili krugova; projektovanje kompjuterskog čipa može da zahteva rad sa milionima elemenata. Svi ovi problemi sadrže obradu geometrijskih objekata. Pošto veličina ulaza za ove probleme može biti vrlo velika, veoma je značajno razviti efikasne algoritme za njihovo rešavanje.

U nastavku ćemo razmotriti nekoliko osnovnih geometrijskih algoritama — onih koji se mogu koristiti kao elementi za izgradnju složenijih algoritama.

Objekti sa kojima se radi su tačke, prave, duži i mnogouglovi. Algoritmi obrađuju ove objekte, odnosno izračunavaju neke njihove karakteristike. Najpre ćemo dati osnovne definicije i navesti strukture podataka pogodne za predstavljanje pojedinih objekata. *Tačka*  $P$  u ravni predstavlja se kao par koordinata  $(x, y)$  (pretpostavlja se da je izabran fiksirani koordinatni sistem). *Prava* je predstavljena parom tačaka  $P$  i  $Q$  (proizvoljne dve različite tačke na pravoj). *Duž* se takođe zadaje parom tačaka  $P$  i  $Q$  koje predstavljaju krajeve te duži, i označavamo je sa  $PQ$ . *Put*  $P$  je niz tačaka  $P_1, P_2, \dots, P_k$  i duži  $P_1P_2, P_2P_3, \dots, P_{k-1}P_k$  koje ih povezuju. Duži koje čine put su njegove *ivice* (*stranice*). *Zatvoreni put* je put čija se poslednja tačka poklapa sa prvom. Zatvoreni put zove se i *mnogougao*. Tačke koje definišu mnogougao su njegova *temena*. Na primer, trougao je mnogougao sa tri temena. Mnogougao se predstavlja nizom, a ne skupom tačaka, jer je bitan redosled kojim se tačke zadaju (promenom redosleda tačaka iz istog skupa u opštem slučaju dobija se drugi mnogougao). *Prosti mnogougao* je onaj kod koga odgovarajući put nema preseka sa samim sobom; drugim rečima, jedine ivice koje imaju zajedničke tačke su susedne ivice sa njihovim zajedničkim temenom. Prosti mnogougao ograničava jednu oblast u ravni, *unutrašnjost* mnogougla. *Konveksni mnogougao* je mnogougao čija unutrašnjost sa svake dve tačke koje sadrži, sadrži i sve tačke te duži (ekvivalentno, mnogougao je konveksan ako su mu svi unutrašnji uglovi manji ili jednaki  $180^\circ$ ). *Konveksni put* je put sastavljen od tačaka  $P_1, P_2, \dots, P_k$  takav da je mnogougao  $P_1P_2 \dots P_k$  konveksan.

Česta neugodna karakteristika geometrijskih problema je postojanje mnogih “specijalnih slučajeva”. Na primer, dve prave u ravni obično se seku u jednoj tački, sem ako su paralelne ili se poklapaju. Pri rešavanju problema sa dve prave, moraju se predvideti sva tri moguća slučaja. Složeniji objekti prouzrokuju pojavu mnogo većeg broja specijalnih slučajeva, o kojima treba voditi računa. Obično se većina tih specijalnih slučajeva neposredno rešava, ali potreba da se oni uzmu u obzir čini ponekad konstrukciju geometrijskih algoritama vrlo iscrpljujućom. Mi

ćemo ponekad ignorisati detalje koji nisu od suštinskog značaja za razumevanje osnovnih ideja algoritama, ali čitaocu savetujemo da razmotri rešavanje svakog od specijalnih slučajeva na koji se pri konstrukciji algoritma naiđe.

## Osnovni algoritmi

Pretpostavlja se da je čitalac upoznat sa osnovama analitičke geometrije. U nastavku ćemo razmotriti neka osnovna pitanja na koja se nailazi prilikom konstrukcije gotovo svakog geometrijskog algoritma, kao što je recimo izračunavanje rastojanja između dve tačke, ispitivanje kolinearnosti tri tačke ili izračunavanje presečne tačke dveju duži. Sve ove operacije mogu se izvesti u konstantnom vremenu, korišćenjem osnovnih aritmetičkih operacija. Pri tome, na primer, pretpostavljamo da se kvadratni koren može izračunati za konstantno vreme.

### Skalarni i vektorski proizvod

Data su dva vektora  $\vec{a}(a_1, \dots, a_n)$  i  $\vec{b}(b_1, \dots, b_n)$  iste dimenzije. Njihov *skalarni proizvod* je skalar čiju vrednost računamo po formuli:

$$\vec{a} \circ \vec{b} = \sum_{i=1}^n a_i \cdot b_i$$

Ukoliko su koordinate vektora celobrojne, i vrednost skalarnog proizvoda biće celobrojna.

```
// funkcija koja racuna skalarni proizvod dva vektora
int skalarniProizvod(vector<int> a, vector<int> b){

    // racunamo dimenziju vektora
    int n = a.size();
    // racunamo skalarni proizvod
    int proizvod = 0;
    for (int i = 0; i < n; i++)
        proizvod += a[i]*b[i];
    return proizvod;
}

int main(){

    vector<int> a = {2,2};
    vector<int> b = {3,4};
    cout << "Skalarni proizvod vektora a i b je: "
         << skalarniProizvod(a,b) << endl;
```

```

    return 0;
}

```

U daljem tekstu ćemo razmatrati vektore određene tačkama u ravni, te će dimenzija vektora biti jednaka 2. Skalarni proizvod vektora  $\vec{a}(a_x, a_y)$  i vektora  $\vec{b}(b_x, b_y)$  jednak je  $\vec{a} \circ \vec{b} = a_x \cdot b_x + a_y \cdot b_y$ .

Važi naredna formula:

$$a \circ b = |a| \cdot |b| \cdot \cos \alpha$$

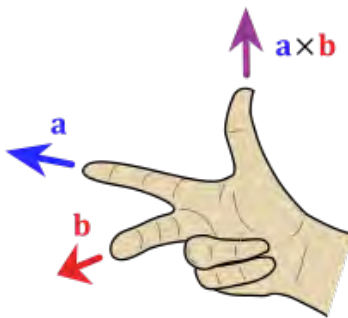
gde je sa  $|a|$  označen intenzitet vektora  $a$ , a sa  $\alpha$  ugao koji zaklapaju ova dva vektora. Na osnovu ove formule moguće je izračunati:

- intenzitet, odnosno dužinu vektora  $\vec{u}$ , na osnovu formule  $|\vec{u}|^2 = \vec{u} \circ \vec{u}$ , jer je  $\cos 0 = 1$ .
- ugao između vektora  $\vec{u}$  i  $\vec{v}$ , na osnovu formule  $\alpha = \arccos \frac{\vec{u} \circ \vec{v}}{|\vec{u}| \cdot |\vec{v}|}$

*Vektorski proizvod* vektora  $\vec{a}(a_x, a_y, a_z)$  i  $\vec{b}(b_x, b_y, b_z)$  dimenzije 3 je vektor ortogonalan na ravan određenu vektorima  $\vec{a}$  i  $\vec{b}$ , čiji je smer određen pravilom desne ruke, a intenzitet jednak površini paralelograma koji određuju vektori  $\vec{a}$  i  $\vec{b}$ , odnosno može se izračunati po formuli:

$$|\vec{a} \times \vec{b}| = |a| \cdot |b| \cdot \sin \alpha$$

gde je sa  $\alpha$  označen manji od uglova koji obrazuju vektori  $\vec{a}$  i  $\vec{b}$ .



Slika 1: Pravilo desne ruke.

Neka je  $\vec{a} = a_x \cdot \vec{i} + a_y \cdot \vec{j} + a_z \cdot \vec{k}$  i  $\vec{b} = b_x \cdot \vec{i} + b_y \cdot \vec{j} + b_z \cdot \vec{k}$ , gde su sa  $\vec{i}$ ,  $\vec{j}$  i  $\vec{k}$  označeni jedinični vektori u smeru  $x$ ,  $y$  i  $z$  koordinatne ose. Važi  $\vec{i} \times \vec{j} = \vec{k} = -\vec{j} \times \vec{i}$ ,  $\vec{j} \times \vec{k} = \vec{i} = -\vec{k} \times \vec{j}$ ,  $\vec{k} \times \vec{i} = \vec{j} = -\vec{i} \times \vec{k}$ , odakle dobijamo da je vektorski proizvod vektora  $\vec{a}$  i  $\vec{b}$  jednak:

$$\vec{a} \times \vec{b} = (a_y b_z - a_z b_y) \vec{i} + (a_z b_x - a_x b_z) \vec{j} + (a_x b_y - a_y b_x) \vec{k}$$

Ova formula se može zapisati u obliku naredne determinante:

$$\vec{a} \times \vec{b} = \begin{vmatrix} \vec{i} & \vec{j} & \vec{k} \\ a_x & a_y & a_z \\ b_x & b_y & b_z \end{vmatrix}$$

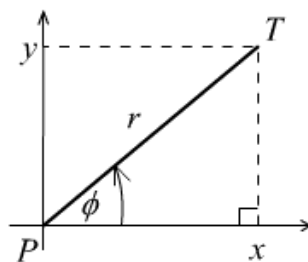
```
// funkcija koja racuna vektorski proizvod p vektora a i b
// u trodimenzionom prostoru
void vektorskiProizvod(vector<int> a, vector<int> b, vector<int>& p){
    p[0] = a[1]*b[2] - a[2]*b[1];
    p[1] = a[2]*b[0] - a[0]*b[2];
    p[2] = a[0]*b[1] - a[1]*b[0];
}

int main(){

    vector<int> a = {3,3,-4};
    vector<int> b = {1,-2,5};
    vector<int> proizvod(3);
    vektorskiProizvod(a,b,proizvod);
    cout << "Vektorski proizvod vektora a i b je: (" ;
    for(int i = 0; i < 3; i++)
        cout << proizvod[i] << " ";
    cout << ")" << endl;
    return 0;
}
```

## Polarne koordinate

Nekada je za date dekartovske koordinate  $x$  i  $y$  tačke  $T$  potrebno odrediti njene polarne koordinate: rastojanje  $r$  od koordinatnog početka  $P$  i ugao  $\phi$  koji vektor  $PT$  zahvata sa pozitivnim delom  $x$  ose i obratno, na osnovu polarnih koordinata odrediti dekartovske.



Slika 2: Veza između dekartovskih i polarnih koordinata.

Da bismo transformisali polarne koordinate u dekartovske, možemo iskoristiti sledeću vezu (slika 2):

$$x = r \cos \phi, \quad y = r \sin \phi$$

A za transformaciju dekartovskih koordinata u polarne, možemo iskoristiti naredne jednačine:

$$r = \sqrt{x^2 + y^2}, \quad \phi = \arctan \frac{y}{x}$$

```
// transformacija dekartovskih koordinata u polarne
void dekartovskeUPolarne(double x, double y, double &r, double &ugao){

    r = sqrt((pow(x,2))+(pow(y,2)));
    ugao = atan2(y,x);
}

// transformacija polarnih u dekartovske koordinate
void polarneUDekartovske(double r, double ugao, double &x, double &y){

    x = r * cos(ugao);
    y = r * sin(ugao);
}

int main(){

    double x, y, r, ugao;
    cout << "Unesi dekartovske koordinate tacke" << endl;
    cin >> x >> y;
    // transformisemo dekartovske koordinate polazne tacke
    // u polarne koordinate
    dekartovskeUPolarne(x,y,r,ugao);
    cout << "Polarne koordinate te tacke su r: "
        << r << " ugao: " << ugao << endl;
    double x1, y1;
    // transformisemo dobijene polarne koordinate u dekartovske
    // proveravamo da li smo dobili polazne koordinate
    polarneUDekartovske(r,ugao,x1,y1);
    cout << "Dekartovske koordinate te tacke su x:"
        << x1 << " y: " << y1 << endl;
    return 0;
}
```

### Površina trougla

**Problem:** Dat je trougao koordinatama svojih temena. Izračunati njegovu površinu.

Zadatak je matematički moguće rešiti na nekoliko načina. Jedan način je da primenimo Heronov obrazac koji kaže da je

$$P = \sqrt{s \cdot (s - a) \cdot (s - b) \cdot (s - c)}$$

gde su sa  $a$ ,  $b$  i  $c$  označene dužine stranica trougla, a sa  $s = (a + b + c)/2$  njegov poluobim. Dužine stranica možemo izračunati kao rastojanje između temena trougla.

Tačku ćemo nadalje predstavljati strukturom podataka koja sadrži dve komponente:  $x$  i  $y$  koordinatu koje su predstavljene realnim brojevima. Alternativno, tačku bismo mogli da predstavimo uređenim parom dva realna broja `pair<double,double>`.

```
// tacku u ravni predstavljamo njenom x i y koordinatom
struct Tacka{
    double x;
    double y;
};

// rastojanje izmedju tacaka
double rastojanje(Tacka A, Tacka B) {
    double dx = B.x - A.x;
    double dy = B.y - A.y;
    return sqrt(dx*dx + dy*dy);
}

// površina trougla za zadate dužine stranica,
// izračunata koriscenjem Heronovog obrasca
double površinaTrougla(double a, double b, double c) {
    double s = (a + b + c) / 2;
    return sqrt(s*(s-a)*(s-b)*(s-c));
}

int main() {

    Tacka A, B, C;
    cout << "Unesi koordinate tacaka A, B i C" << endl;
    cin >> A.x >> A.y >> B.x >> B.y >> C.x >> C.y;

    // racunamo dužine stranica trougla
    double a = rastojanje(B, C);
    double b = rastojanje(A, C);
    double c = rastojanje(A, B);

    // racunamo površinu trougla
    double P = površinaTrougla(a, b, c);
```

```

    cout << "Povrsina trougla je: " << P << endl;
    return 0;
}

```

Drugi način da dođemo do površine jeste primenom činjenice da je intenzitet vektorskog proizvoda vektora  $\vec{AB}$  i  $\vec{AC}$  jednak površini paralelograma koji oni obrazuju. Pošto je naš trougao upravo polovina tog paralelograma njegova površina jednaka je polovini intenziteta vektorskog proizvoda. Koordinate  $x$  i  $y$  vektora  $\vec{AB}$  i  $\vec{AC}$  moguće je izračunati oduzimanjem koordinata tačke  $A$  od tačaka  $B$  i  $C$ . Pošto možemo pretpostaviti da naš trougao leži u ravni  $xOy$ , koordinata  $z$  ovih vektora jednaka je 0. Vektorski proizvod dva vektora čije su koordinate poznate jednak je determinanti:

$$\begin{vmatrix} \vec{i} & \vec{j} & \vec{k} \\ a_x & a_y & a_z \\ b_x & b_y & b_z \end{vmatrix} = (a_y b_z - a_z b_y, a_z b_x - a_x b_z, a_x b_y - a_y b_x)$$

Jedan način da se napravi implementacija je da se iskoristi funkcija koja izračunava vektorski proizvod proizvoljna dva vektora dimenzije 3.

```

// duzina (intenzitet) vektora u
double duzinaVektora(vector<double> u) {
    return sqrt(u[0]*u[0] + u[1]*u[1] + u[2]*u[2]);
}

// funkcija koja racuna vektorski proizvod p vektora a i b
// u trodimenzionom prostoru
void vektorskiProizvod(vector<double> a, vector<double> b,
                      vector<double>& p){
    p[0] = a[1]*b[2] - a[2]*b[1];
    p[1] = a[0]*b[2] - a[2]*b[0];
    p[2] = a[0]*b[1] - a[1]*b[0];
}

// površina trougla zadanog temenima A, B i C
double površinaTrougla(Tacka A, Tacka B, Tacka C) {

    // vektor AB
    vector<double> AB;
    AB.push_back(B.x-A.x);
    AB.push_back(B.y-A.y);
    AB.push_back(0);

    // vektor AC
    vector<double> AC;
    AC.push_back(C.x-A.x);

```

```

AC.push_back(C.y-A.y);
AC.push_back(0);

// vektorski proizvod v = AB x AC
vector<double> v(3);
vektorskiProizvod(AB, AC, v);

// površina je polovina intenziteta vektorskog proizvoda
return duzinaVektora(v) / 2.0;
}

int main() {

    Tacka A, B, C;
    cout << "Unesi koordinate tacaka A, B i C" << endl;
    cin >> A.x >> A.y >> B.x >> B.y >> C.x >> C.y;

    // površina trougla
    double P = površinaTrougla(A, B, C);
    cout << "Površina trougla je: " << P << endl;

    return 0;
}

```

Pošto je u ovoj situaciji potrebno izračunati vektorski proizvod dva vektora koji leže u ravni  $xOy$ , tj. dva vektora kojima su  $z$  koordinate jednake nuli, izračunavanje je moguće optimizovati. Naime, rezultat će biti vektor normalan na ravan  $xOy$ , tj. koordinate  $x$  i  $y$  će mu biti jednake nuli. Zato će mu intenzitet biti jednak apsolutnoj vrednosti koordinate  $z$  koja je jednaka  $x_1y_2 - y_1x_2$ , pri čemu su  $(x_1, y_1)$  koordinate vektora  $\vec{AB}$ , a  $(x_2, y_2)$  koordinate vektora  $\vec{AC}$ . Dakle, površina trougla biće jednaka

$$\frac{|(b_x - a_x)(c_y - a_y) - (b_y - a_y)(c_x - a_x)|}{2}$$

tj.

$$\frac{|b_x c_y - a_x c_y - b_x a_y - b_y c_x + a_y c_x + a_x b_y|}{2}$$

```

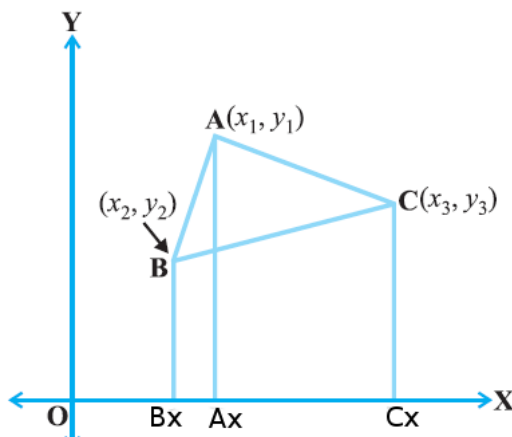
double površinaTrougla(Tacka A, Tacka B, Tacka C) {
    return abs(B.x*C.y - A.x*C.y - B.x*A.y
        - B.y*C.x + A.y*C.x + A.x*B.y)/2.0;
}

```

Napomenimo da je rešenje zasnovano na vektorskom proizvodu mnogo bolje nego ono zasnovano na Heronovom obrascu, jer koristi samo osnovne aritmetičke operacije (a ne i korenovanje).



Treći način dolaska do rešenja je da se površina trougla iskaže kao razlika površina određenih trapeza. Na primer, ako važi raspored  $a_x < b_x < c_x$ , i ako su  $A_x$ ,  $B_x$  i  $C_x$  redom projekcije tačaka  $A$ ,  $B$  i  $C$  na  $x$ -osu, tada je površina trougla jednaka razlici između zbira površina pravouglavih trapeza  $AA_xB_xB$ ,  $BB_xC_xC$  i površine pravouglavih trapeza  $AA_xC_xC$ .



Slika 3: Površina trougla izražena preko površina trapeza.

Površina trapeza  $AA_xB_xB$  jednaka je proizvodu dužine njegove srednje linije i dužine njegove visine. Dužina osnovice  $A_xA$  jednaka je apsolutnoj vrednosti koordinate  $a_y$ , dužina osnovice  $B_xB$  jednaka je apsolutnoj vrednosti koordinate  $b_y$  dok je dužina visine jednaka apsolutnoj vrednosti razlike koordinata  $b_x$  i  $a_x$ . Tako da je površina tog trapeza jednaka  $\frac{|b_x - a_x|(|a_y| + |b_y|)}{2}$ . Na sličan način mogu se izvesti i formule za druga dva trapeza.

Analizom mogućih rasporeda tačaka, može se pokazati da će se ispravan rezultat dobiti i ako se posmatra takozvana označena površina trapeza, koja dopušta negativne dužine i površine. Označena površina trougla biće jednaka zbiru označenih površina tri pomenuta pravouglavi trapeza ( $AA_xB_xB$ ,  $BB_xC_xC$  i  $CC_xA_xA$ ), a prava površina trougla biće jednaka njenoj apsolutnoj vrednosti. Označene površine ovih trapeza dobijaju se tako što se u prethodno izvedenim formulama za njihovu površinu zanemare apsolutne vrednosti i jednake su  $\frac{(b_x - a_x)(b_y + a_y)}{2}$ ,  $\frac{(c_x - b_x)(c_y + b_y)}{2}$  i  $\frac{(a_x - c_x)(a_y + c_y)}{2}$ , tako da se površina trougla dobija formulom

$$\frac{|(b_x - a_x)(b_y + a_y) + (c_x - b_x)(c_y + b_y) + (a_x - c_x)(a_y + c_y)|}{2}$$

```
// racuna oznacenu površinu pravouglavih trapeza koji određuju tačke
// M, N i njihove projekcije na x-osu
double površinaTrapeza(Tacka M, Tacka N) {
    return (N.x - M.x) * (M.y + N.y);
}
```

```

}

// površina trougla zadatog temenima A, B, C
double površinaTrougla(Tacka A, Tacka B, Tacka C) {
    return abs(površinaTrapeza(A, B) +
               površinaTrapeza(B, C) +
               površinaTrapeza(C, A)) / 2.0;
}

```

Gornji izraz možemo srediti i dobiti da je površina trougla jednaka:

$$\frac{|b_x a_y - a_x b_y + c_x b_y - b_x c_y + a_x c_y - c_x a_y|}{2}$$

Primetimo da smo na ovaj način dobili potpuno istu formulu kao kada smo površinu trougla računali vektorskim proizvodom. Ova formula se nekada naziva *pravilo pertle*. Zaista, ako napišemo koordinate tačaka u sledećem obliku

$$\begin{array}{cc} a_x & a_y \\ b_x & b_y \\ c_x & c_y \\ a_x & a_y \end{array}$$

formula se gradi tako što se sa jednim znakom uzimaju proizvodi odozgo-naniže, sleva-udesno (to su  $a_x b_y$ ,  $b_x c_y$  i  $c_x a_y$ ), dok se sa suprotnim znakom uzimaju proizvodi odozdo-naviše, sleva udesno (to su  $a_x c_y$ ,  $c_x b_y$  i  $b_x a_y$ ), što, kada se nacрта, zaista podseća na cik-cak vezivanje pertli.

```

double površinaTrougla(Tacka A, Tacka B, Tacka C) {
    // pertle (cik-cak)
    return abs(A.x*B.y + B.x*C.y + C.x*A.y
               - A.x*C.y - C.x*B.y - B.x*A.y) / 2.0;
}

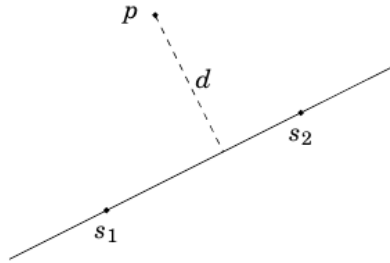
```

## Rastojanje tačke od prave

**Problem:** Odrediti rastojanje  $d$  tačke  $P$  od prave određene tačkama  $S_1$  i  $S_2$ .

Površina trougla sa temenima  $P$ ,  $S_1$  i  $S_2$  može se izračunati na dva načina: ona je jednaka  $\frac{|S_1 S_2| \cdot d}{2}$  i istovremeno je jednaka  $\frac{|PS_1 \times PS_2|}{2}$ . Dakle, rastojanje možemo izračunati na osnovu jednačine:

$$d = \frac{|PS_1 \times PS_2|}{|S_1 S_2|}$$



Slika 4: Rastojanje tačke od prave.

### Kolinearnost tačaka

**Problem:** Date su tri tačke svojim koordinatama  $A(a_x, a_y)$ ,  $B(b_x, b_y)$  i  $C(c_x, c_y)$ . Utvrditi da li su one kolinearne.

Tri tačke su kolinearne ukoliko je nagib prave određene tačkama  $A$  i  $B$  jednak nagibu prave određene tačkama  $A$  i  $C$ . Nagib prave određene tačkama  $A$  i  $B$  jednak je količniku razlike njihovih  $y$  i  $x$  koordinata:  $k_{AB} = \frac{b_y - a_y}{b_x - a_x}$ . Slično,  $k_{AC} = \frac{c_y - a_y}{c_x - a_x}$ . Dakle, tačke će biti kolinearne ukoliko važi:

$$\frac{b_y - a_y}{b_x - a_x} = \frac{c_y - a_y}{c_x - a_x}$$

odnosno:

$$(b_y - a_y) \cdot (c_x - a_x) = (c_y - a_y) \cdot (b_x - a_x)$$

```
// ispituje se kolinearnost tacaka
// razmatranjem nagiba po dve tacke
void kolinearne(Tacka A, Tacka B, Tacka C){
    if ((B.y - A.y) * (C.x - A.x) == (C.y - A.y) * (B.x - A.x))
        cout << "Kolinearne su" << endl;
    else
        cout << "Nisu kolinearne" << endl;
}

int main(){

    Tacka A, B, C;
    cout << "Unesi koordinate tacaka A, B i C" << endl;
    cin >> A.x >> A.y >> B.x >> B.y >> C.x >> C.y;
    kolinearne(A, B, C);
    return 0;
}
```

Primetimo da smo prilikom ispitivanja kolinearnosti tačaka vrednosti određenih izraza poredili na jednakost operatorom `==`. Ovo bi bilo korektno u slučaju da su koordinate tačaka (a samim tim i vrednosti koje poredimo) celobrojne. Ukoliko su koordinate realni brojevi, onda bi poređenje na jednakost trebalo implementirati proverom da li je apsolutna vrednost razlike vrednosti datih izraza manja od vrednosti  $\epsilon$  gde je  $\epsilon$  neka mala pozitivna vrednost.

Drugi pristup rešavanju ovog problema bi se zasnivao na narednom tvrđenju: tri tačke pripadaju istoj pravoj ako je površina trougla određenog ovim trima tačkama jednaka 0. Na osnovu prethodnog razmatranja, znamo da se površina trougla  $ABC$  može izračunati po formuli:

$$\frac{|b_x a_y - a_x b_y + c_x b_y - b_x c_y + a_x c_y - c_x a_y|}{2}$$

Ukoliko su koordinate tačaka celobrojne, vrednost  $2P$  je takođe celobrojna i jednaka je 0 ako su tačke kolinearne, te ćemo u narednoj implementaciji pretpostaviti da su temena trougla celobrojna i računati dvostruku označenu površinu trougla (bez računanja apsolutne vrednosti).

```
struct Tacka{
    int x;
    int y;
};

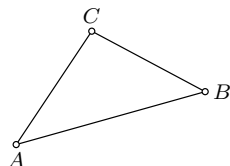
// funkcija za izracunavanje dvostruke oznacene površine trougla
// ako su zadata njegova temena
int dvostrukaPovrsina(Tacka A, Tacka B, Tacka C){
    return A.x*B.y + B.x*C.y + C.x*A.y
        - A.x*C.y - C.x*B.y - B.x*A.y;
}

// funkcija koja utvrdjuje da li su tacke kolinearne ili ne
void kolinearne(Tacka A, Tacka B, Tacka C){
    int P = dvostrukaPovrsina(A, B, C);
    if (P == 0)
        cout << "Kolinearne su";
    else
        cout << "Nisu kolinearne";
}
```

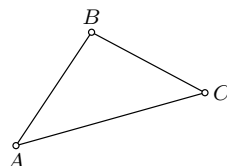
## Orijentacija

Uređena trojka tačaka u ravni može biti kolinearna ili imati:

- orijentaciju u smeru suprotnom od kretanja kazaljke na časovniku (pozitivna orijentacija)



Slika 5: Trougao  $ABC$  ima matematički pozitivnu orijentaciju.



Slika 6: Trougao  $ABC$  ima matematički negativnu orijentaciju.

- orijentaciju u smeru kretanja kazaljke na časovniku (negativna orijentacija)

Važe naredna svojstva:

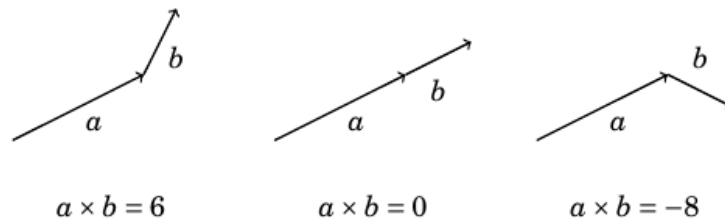
- ako uređena trojka  $A, B, C$  ima pozitivnu (negativnu) orijentaciju, onda i uređena trojka  $B, C, A$  ima pozitivnu (negativnu) orijentaciju
- ako uređena trojka  $A, B, C$  ima pozitivnu (negativnu) orijentaciju, onda uređena trojka  $B, A, C$  ima negativnu (pozitivnu) orijentaciju

**Problem:** Za dati trougao  $ABC$  odrediti orijentaciju.

Ako razmotrimo vektore  $a$  i  $b$  koji su nadovezani jedan na drugi, njihov vektorski proizvod nam daje informaciju da li vektor  $b$  pravi zaokret ulevo (pozitivna vrednost), ne skreće ni na jednu stranu (vrednost nula) ili pravi zaokret udesno (negativna vrednost) kada se postavi direktno nakon  $a$ .

Važi naredno tvrđenje: trougao  $ABC$  ima pozitivnu orijentaciju ako vektorski proizvod vektora  $\vec{AB}$  i  $\vec{AC}$  ima pozitivnu vrednost.

S obzirom na to da temena  $A, B$  i  $C$  pripadaju ravni  $Oxy$ , i vektori  $\vec{AB}$  i  $\vec{AC}$  pripadaće ovoj ravni, odnosno imaju koordinate  $AB(x_1, y_1, 0)$  i  $AC(x_2, y_2, 0)$ . Njihov vektorski proizvod biće vektor upravan na ravan  $Oxy$ , odnosno vektor sa koordinatama  $(0, 0, z)$ . Stoga će njegov intenzitet biti jednak apsolutnoj vrednosti koordinate  $z$  koja je jednaka  $x_1y_2 - y_1x_2$ .



Slika 7: Različiti položaji vektora vode različitoj orijentaciji.

Dakle, ako su koordinate temena  $A(a_x, a_y, 0)$ ,  $B(b_x, b_y, 0)$  i  $C(c_x, c_y, 0)$  da bismo odredili orijentaciju trougla  $ABC$  interesuje nas znak izraza:

$$d = (b_x - a_x)(c_y - a_y) - (c_x - a_x)(b_y - a_y)$$

Ukoliko je:

- $d > 0$ : trougao ima pozitivnu orijentaciju
- $d < 0$ : trougao ima negativnu orijentaciju
- $d = 0$ : tačke su kolinearne

```
enum orij {KOLINEARNE,NEGATIVNA_ORJ,POZITIVNA_ORJ};

// funkcija koja utvrđuje orijentaciju tacaka P1, P2 i P3
orij orijentacija(Tacka P1, Tacka P2, Tacka P3){
    int d = (P2.x-P1.x)*(P3.y-P1.y) - (P3.x-P1.x)*(P2.y-P1.y);
    if (d == 0) return KOLINEARNE;
    else if (d > 0)
        return POZITIVNA_ORJ;
    else return NEGATIVNA_ORJ;
}

int main(){

    Tacka A, B, C;
    cout << "Unesi koordinate tacaka A, B i C" << endl;
    cin >> A.x >> A.y >> B.x >> B.y >> C.x >> C.y;

    orij o = orijentacija(A, B, C);
    if (o == KOLINEARNE)
        cout << "Tacke A, B i C su kolinearne" << endl;
    else if (o == NEGATIVNA_ORJ)
        cout << "Trougao ABC ima negativnu orijentaciju" << endl;
    else
```

```

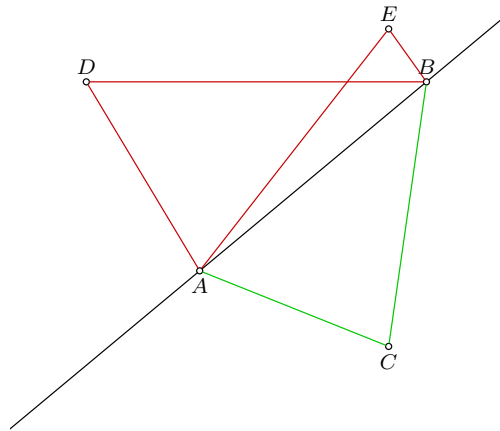
        cout << "Trogao ABC ima pozitivnu orijentaciju" << endl;
    return 0;
}

```

I ovde se prilikom ispitivanja orijentacije tačaka podrazumeva da su koordinate celobrojne, te se proverava da li su vrednosti dva izraza jednake svodi na primenu operatora `==`. Ukoliko tačke imaju realne koordinate, i odgovarajući izrazi bi bili realne vrednosti, te bi proveru jednakosti trebalo drugačije realizovati.

**Problem:** Za date tačke  $C$  i  $D$  utvrditi da li su sa iste strane prave  $p$  određene tačkama  $A$  i  $B$ .

Tačke  $C$  i  $D$  biće sa iste strane prave  $p$  ako i samo ako su trouglovi  $ABC$  i  $ABD$ ,  $A, B \in p$  iste orijentacije. Dakle, dovoljno je da ispitamo da li su odgovarajući vektorski proizvodi istog znaka.



Slika 8: Tačke koje se nalaze sa iste strane/sa različitih strana prave.

```

// funkcija za izracunavanje dvostruke oznacene površine trougla
// ako su zadata njegova temena
int dvostrukaPovrsina(Tacka A, Tacka B, Tacka C){
    return A.x*B.y + B.x*C.y + C.x*A.y
}

```

```

        - A.x*C.y - C.x*B.y - B.x*A.y;
    }

    int main() {

        Tacka A, B, C, D;
        cout << "Unesi koordinate tacaka A i B" << endl;
        cin >> A.x >> A.y >> B.x >> B.y;
        cout << "Unesi koordinate tacaka C i D" << endl;
        cin >> C.x >> C.y >> D.x >> D.y;

        // površina trougla
        int P1 = dvostrukaPovrsina(A,B,C);
        int P2 = dvostrukaPovrsina(A,B,D);
        if ((P1 > 0 && P2 > 0) || (P1 < 0 && P2 < 0))
            cout << "Tacke se nalaze sa iste strane" << endl;
        else
            cout << "Tacke se nalaze sa razlicitih strana" << endl;
        return 0;
    }

```

**Problem:** Date su dve duži  $P_1Q_1$  i  $P_2Q_2$  u ravni. Utvrditi da li se ove dve duži seku.

Važi sledeće tvrđenje: Duži  $P_1Q_1$  i  $P_2Q_2$  se seku ako i samo ako važi jedan od sledećih uslova:

1. trojke tačaka  $P_1, Q_1, P_2$  i  $P_1, Q_1, Q_2$  imaju različitu orijentaciju i trojke tačaka  $P_2, Q_2, P_1$  i  $P_2, Q_2, Q_1$  imaju različitu orijentaciju (razmatramo tri moguće različite vrednosti orijentacije – kolinearnost, pozitivna i negativna orijentacija)
2. sve četiri navedene trojke tačaka su kolinearne i seku se i projekcije duži na  $x$  osu i projekcije duži na  $y$  osu.

```

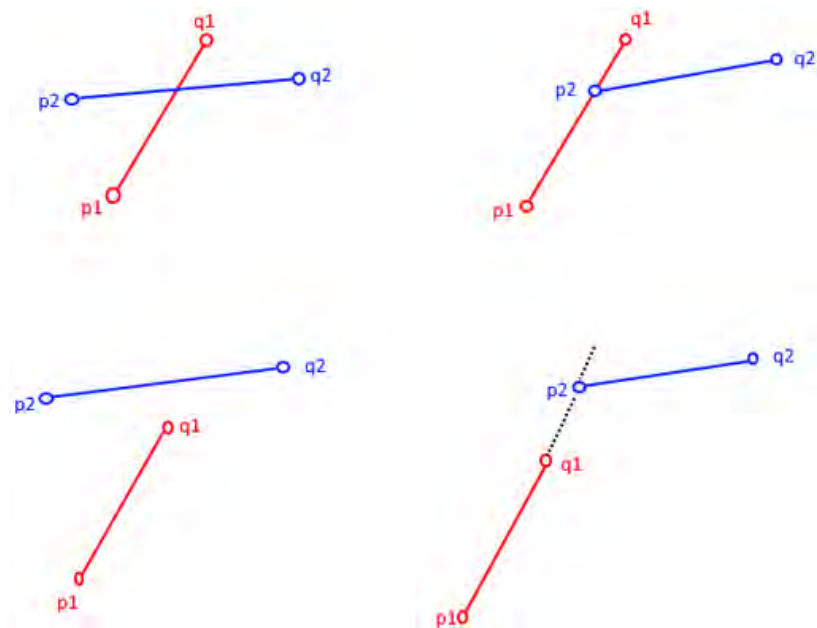
// za date tri kolinearne tacke P, Q i R proveravamo da li
// tacka Q pripada duzi PR
bool pripadaDuzi(Tacka P, Tacka Q, Tacka R){

    if (Q.x <= max(P.x, R.x) && Q.x >= min(P.x, R.x) &&
        Q.y <= max(P.y, R.y) && Q.y >= min(P.y, R.y))
        return true;
    return false;
}

// funkcija koja utvrdjuje orijentaciju tacaka P1, P2 i P3
int orijentacija(Tacka P1, Tacka P2, Tacka P3){
    int d = (P2.x-P1.x)*(P3.y-P1.y) - (P3.x-P1.x)*(P2.y-P1.y);
}

```





Slika 9: Različiti položaji duži u ravni u slučaju kada nisu sve četiri krajnje tačke duži kolinearne.



Slika 10: Različiti položaji duži u prostoru u slučaju kada su sve četiri krajnje tačke duži kolinearne.

```

    if (d == 0) return KOLINEARNE;
    else if (d > 0)
        return POZITIVNA_ORJ;
    else return NEGATIVNA_ORJ;
}

// proveravamo da li se duzi P1Q1 i P2Q2 seku
bool sekuSe(Tacka P1, Tacka Q1, Tacka P2, Tacka Q2){

    // racunamo vrednosti cetiri orijentacije
    // koje su nam potrebne
    orij o1 = orijentacija(P1, Q1, P2);
    orij o2 = orijentacija(P1, Q1, Q2);
    orij o3 = orijentacija(P2, Q2, P1);
    orij o4 = orijentacija(P2, Q2, Q1);

    // prvi slucaj
    if (o1 != o2 && o3 != o4)
        return true;

    // drugi slucaj
    // P1, Q1 i P2 su kolinearne i P2 pripada duzi P1Q1
    if (o1 == KOLINEARNE && pripadaDuzi(P1, P2, Q1))
        return true;

    // P1, Q1 i Q2 su kolinearne i Q2 pripada duzi P1Q1
    if (o2 == KOLINEARNE && pripadaDuzi(P1, Q2, Q1))
        return true;

    // P2, Q2 i P1 su kolinearne i P1 pripada duzi P2Q2
    if (o3 == KOLINEARNE && pripadaDuzi(P2, P1, Q2))
        return true;

    // P2, Q2 i P1 su kolinearne i Q1 pripada duzi P2Q2
    if (o4 == KOLINEARNE && pripadaDuzi(P2, Q1, Q2))
        return true;

    // inace se duzi ne seku
    return false;
}

int main(){

    Tacka P1, Q1, P2, Q2;
    cout << "Unesi koordinate tacaka P1 i Q1" << endl;
    cin >> P1.x >> P1.y >> Q1.x >> Q1.y;
}

```

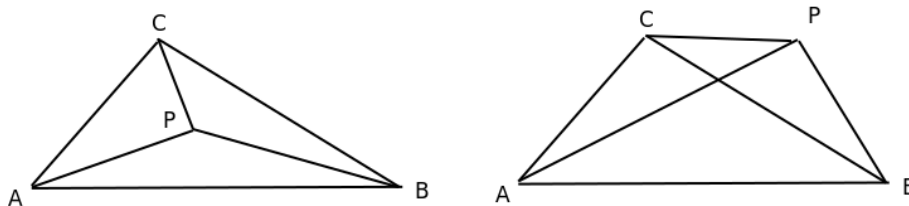
```

    cout << "Unesi koordinate tacaka P2 i Q2" << endl;
    cin >> P2.x >> P2.y >> Q2.x >> Q2.y;
    if (sekuSe(P1, Q1, P2, Q2))
        cout << "Duzi P1Q1 i P2Q2 se seku" << endl;
    else
        cout << "Duzi P1Q1 i P2Q2 se ne seku" << endl;
    return 0;
}

```

### Utvrđivanje da li data tačka pripada trouglu

**Problem:** Dat je trougao  $ABC$  i tačka  $P$ . Ispitati da li tačka  $P$  pripada trouglu  $ABC$  ili ne.



Slika 11: Situacija kada tačka  $P$  pripada i kada ne pripada trouglu  $ABC$ .

Važi sledeće tvrđenje: ako je tačka  $P$  u unutrašnjosti trougla  $ABC$ , onda je zbir površina trouglova  $PAB$ ,  $PBC$  i  $PCA$  jednak površini trougla  $ABC$ .

```

#define EPS 1.0E-6

// funkcija kojom proveravamo da li se tacka P nalazi u trouglu ABC
bool tackaUTrouglu(Tacka A, Tacka B, Tacka C, Tacka P) {

    // racunamo površinu trougla ABC
    double P = povrsina(A, B, C);

    // racunamo površine trouglova PBC, PAC i PAB
    double P1 = povrsina(P,B,C);
    double P2 = povrsina(A,P,C);
    double P3 = povrsina(A,B,P);

    // proveravamo da li one u zbiru daju površinu trougla ABC
    // poredimo dve realne vrednosti na jednakost
    return fabs(P - (P1 + P2 + P3)) < EPS;
}

int main(){

```

```

Tacka A, B, C, P;
cout << "Unesi koordinate temena A, B i C" << endl;
cin >> A.x >> A.y >> B.x >> B.y >> C.x >> C.y;
cout << "Unesi koordinate tacke P" << endl;
cin >> P.x >> P.y;

if (tackaUTrouglu(A,B,C,P))
    cout << "Tacka se nalazi u trouglu" << endl;
else
    cout << "Tacka se ne nalazi u trouglu" << endl;

return 0;
}

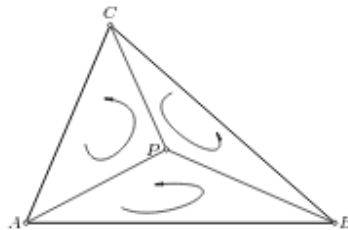
Važi takođe i sledeće: tačka  $P$  se nalazi u trouglu  $ABC$  ako i samo ako su
trouglovi  $ABP$ ,  $BCP$  i  $CAP$  iste orijentacije.

// funkcija kojom proveravamo da li se tacka P nalazi u trouglu ABC
bool tackaUTrouglu(Tacka A, Tacka B, Tacka C, Tacka P){

    // racunamo orijentacije sva tri trougla
    orij o1 = orijentacija(A,B,P);
    orij o2 = orijentacija(B,C,P);
    orij o3 = orijentacija(C,A,P);

    if (o1 == o2 && o2 == o3)
        return true;
    else
        return false;
}

```

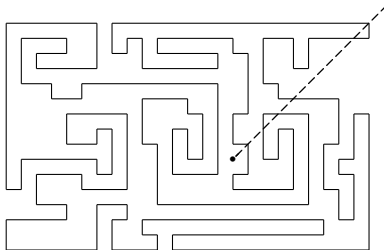


Slika 12: Orijentacije trouglova  $ABP$ ,  $BCP$  i  $CAP$  kada tačka  $P$  pripada trouglu  $ABC$ .

## Utvrđivanje da li zadata tačka pripada mnogouglu

Razmatranje započinjemo jednim jednostavnim problemom.

**Problem** Zadat je prost mnogougao  $P$  i tačka  $Q$ . Ustanoviti da li je tačka  $Q$  u ili van mnogougla  $P$ .



Slika 13: Utvrđivanje pripadnosti tačke unutrašnjosti prostog mnogougla.

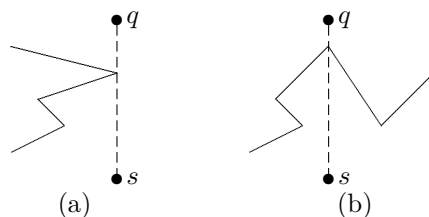
Problem izgleda jednostavno na prvi pogled, ali ako se razmatraju složeni nekonveksni mnogouglovi, kao onaj na slici 13, problem sigurno nije jednostavan. Prvi intuitivni pristup je pokušati nekako “izaći napolje”, polazeći od zadate tačke. Posmatrajmo proizvoljnu polupravu sa temenom  $Q$ . Vidi se da je dovoljno prebrojati preseke sa stranicama mnogougla, sve do dostizanja spoljašnje oblasti. U primeru na slici 13, idući na severoistok od date tačke (prateći isprekidanu liniju), nailazimo na šest preseka sa mnogougлом do dostizanja spoljašnje oblasti (istina, već posle četiri preseka stiže se van mnogougla, ali to računar “ne vidi”; zato se broji ukupan broj preseka poluprave sa stranicama mnogougla). Pošto nas poslednji presek pre izlaska izvodi iz mnogougla, a pretposlednji nas vraća u mnogougao, itd., tačka je van mnogougla. Uopšte tačka je u mnogouglu ako i samo ako je broj preseka neparan.

Razvijajući ovaj algoritam, implicitno smo pretpostavljali da radimo sa slikom. Problem je nešto drugačiji kad je ulaz dat nizom koordinata, što je uobičajeno. Na primer, kad posao radimo ručno i vidimo mnogougao, lako je naći dobar put (onaj sa malo preseka) do neke tačke van mnogougla. U slučaju mnogougla datog nizom koordinata, to nije lako. Najveći deo vremena troši se na izračunavanje preseka. Taj deo posla može se bitno uprostiti ako je duž  $QS$  paralelna jednoj od osa — na primer  $y$ -osi. Broj preseka sa ovom specijalnom duži može biti mnogo veći nego sa optimalnom duži (koju nije lako odrediti — čitaocu se ostavlja da razmotri ovaj problem), ali je nalaženje preseka mnogo jednostavnije (za konstantni faktor).

Kao što je rečeno u prethodnom odeljku, obično postoje neki specijalni slučajevi koje treba posebno razmotriti. Neka je  $S$  tačka van mnogougla, i neka je  $l$  duž koja spaja tačke  $Q$  i  $S$ . Cilj je utvrditi da li tačka  $Q$  pripada unutrašnjosti mnogougla  $P$  na osnovu broja preseka duži  $l$  sa ivicama mnogougla  $P$ . Međutim, duž  $l$  može se delom preklapati sa nekim ivicama mnogougla  $P$ . Ovo preklapanje

očigledno ne treba brojati u preseke. Drugi specijalni slučaj je kada duž  $l$  sadrži neko teme  $P_i$  mnogougla. Na slici 14(a) prikazan je slučaj kad presek prave  $l$  sa temenom ne treba brojati, a na slici 14(b) slučaj kad taj presek treba brojati. Postoji više načina za utvrđivanje da li neki ovakav presek treba brojati ili ne. Na primer:

- ako su dva susedna temena za teme  $P_i$  sa iste strane prave  $l$ , presek se ne računa. U protivnom ako su dva susedna temena sa različitih strana prave  $l$ , presek se broji.
- s obzirom na to da razmatramo pravu koja je paralelna sa  $y$  osom, za svaku ivicu mnogougla trebalo bi uračunati recimo levo teme, a desno ne (temena klasifikujemo kao levo i desno u odnosu na vrednosti  $x$  koordinate). Time nećemo računati presek nalik onom na slici 14(a) jer je za obe ivice presek kroz desno teme, a presek prikazan na slici 14(b) hoćemo jednom, jer je za jednu od ivica koje se susstiču u tom temenu to levo teme, a za drugu desno.



Slika 14: Specijalni slučajevi kad prava seče ivicu kroz teme.

```
// utvrđuje da li se tacka A nalazi u datom mnogouglu M
// racunanjem parnosti preseka sa vertikalnom polupravom
// usmerenom nadole sa temenom u tacki A
bool tackaUMnogouglu(vector<Tacka> M, Tacka A){

    int n = M.size();
    // mnogougao mora da ima bar 3 temena
    if (n < 3)
        return false;

    bool uMnogouglu = 0;

    for(int i = 0; i < n; i++){
        // naredno teme u poretku mnogougla
        int j = (i+1)%n;
        // ukoliko se x koordinata tacke A nalazi izmedju vrednosti
        // x koordinata krajnjih tacaka ivice i presecna tacka je
        // na vertikalnoj polupravoj usmerenoj nanize sa pocetkom u tacki A
        if ( A.x > min(M[i].x,M[j].x) && A.x < max(M[i].x,M[j].x)
            && M[i].y + (M[j].y - M[i].y)/(M[j].x - M[i].x)
```

```

        * (A.x - M[i].x) < A.y)
    // registrujemo jos jedan presek
    uMnogouglu = !uMnogouglu;

}
return uMnogouglu;
}

int main()
{

    vector<Tacka> M = {{0, 0}, {10, 1}, {12, 12}, {2, 10}};
    int n = M.size();
    Tacka A = {5, 8};
    if (tackaUMnogouglu(M, A))
        cout << "Tacka se nalazi u mnogouglu" << endl;
    else
        cout << "Tacka se ne nalazi u mnogouglu" << endl;

    return 0;
}

```

Neka je  $n$  broj ivica mnogougla. Kroz osnovnu petlju algoritma prolazi se  $n$  puta. U svakom prolasku nalazi se presek dve duži i izvršavaju se još neke operacije za konstantno vreme. Dakle, ukupno vreme izvršavanja algoritma je  $O(n)$ .

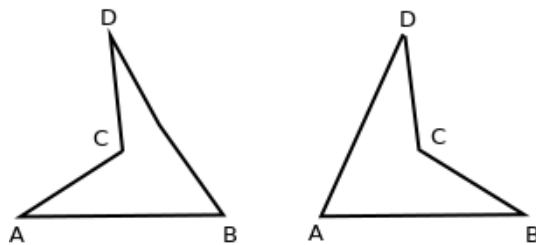
## Konstrukcija prostog mnogougla

Skup tačaka u ravni definiše mnogo različitih mnogouglova, zavisno od izabranog redosleda tačaka. Razmotrićemo sada pronalaženje prostog mnogougla sa zadatim skupom temena.

**Problem:** Dato je  $n$  tačaka u ravni, takvih da nisu sve kolinearne. Povezati ih prostim mnogouglo.

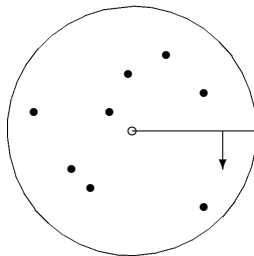
Postoji više metoda za konstrukciju traženog prostog mnogougla; uostalom, jasno je da u opštem slučaju problem nema jednoznačno rešenje (slika 15).

Prikazaćemo najpre geometrijski pristup ovom problemu. Neka je  $C$  neki krug, čija unutrašnjost sadrži sve tačke. Za nalaženje takvog kruga dovoljno je  $O(n)$  operacija — izračunavanja najvećeg među rastojanjima proizvoljne tačke ravni (centra kruga) do svih  $n$  tačaka. Površina  $C$  može se “prebrisati” (pregledati) rotirajućom polupravom kojoj je početak centar  $C$  (slika 16). Pretpostavimo za trenutak da rotirajuća poluprava u svakom trenutku sadrži najviše jednu tačku. Očekujemo da ćemo spajanjem tačaka onim redom kojim poluprava nailazi na njih dobiti prost mnogougao. Pokušajmo da to dokažemo. Označimo tačke, uređene u skladu sa redosledom nailaska poluprave na njih, sa  $P_1, P_2, \dots, P_n$



Slika 15: Tačke  $A$ ,  $B$ ,  $C$  i  $D$  i dva različita prosta mnogougla nad njima.

(prva tačka bira se proizvoljno). Za svako  $i$ ,  $1 \leq i \leq n$ , stranica  $P_i P_{i-1}$  (odnosno  $P_1 P_n$  za  $i = 1$ ) sadržana je u novom (disjunktnom) isečku kruga, pa se ne seče ni sa jednom drugom stranom. Ako bi ovo tvrđenje bilo tačno, dobijeni mnogougao bi morao da bude prost. Međutim, ugao između polupravih kroz neke dve uzastopne tačke  $P_i$  i  $P_{i+1}$  može da bude veći od  $\pi$ . Tada isečak koji sadrži duž  $P_i P_{i+1}$  sadrži više od pola kruga i nije konveksna figura, a duž  $P_i P_{i+1}$  prolazi kroz druge isečke kruga, pa može da seče druge strane mnogougla. Da bismo se uverili da je to moguće, dovoljno je da zamislimo krug sa centrom van kruga sa slike 16. Ovo je dobar primer specijalnih slučajeva na koje se nailazi pri rešavanju geometrijskih problema.

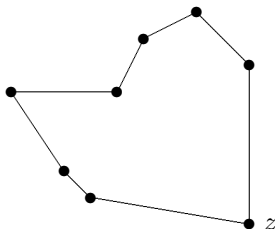


Slika 16: Prolazak tačaka u krugu rotirajućom polupravom.

Da bi se rešio uočeni problem, mogu se, na primer, fiksirati proizvoljne tri tačke iz skupa, a za centar kruga izabrati neka tačka unutar njima određenog trougla (na primer težište, koje se lako nalazi). Ovakav izbor garantuje da ni jedan od dobijenih sektora kruga neće imati ugao veći od  $\pi$ . Moguće je izabrati i drugo rešenje, da se za centar kruga uzme jedna od tačaka iz skupa — tačka  $z$  sa najvećom  $x$ -koordinatom (i sa najmanjom  $y$ -koordinatom, ako ima više tačaka sa najvećom  $x$ -koordinatom). Zatim koristimo isti osnovni algoritam. Sortiramo tačke prema položaju u krugu sa centrom  $z$ . Preciznije, sortiraju se *uglovi* između  $x$ -ose i polupravih od  $z$  ka ostalim tačkama. Ako dve ili više tačaka imaju isti ugao, one se dalje sortiraju prema rastojanju od tačke  $z$ . Na kraju,  $z$  se povezuje sa tačkom sa najmanjim i najvećim uglom, a ostale tačke



povezuju se u skladu sa dobijenim uređenjem, po dve uzastopne. Pošto sve tačke leže levo od  $z$ , do degenerisanog slučaja o kome je bilo reči ne može doći. Prost mnogougao dobijen ovim postupkom za tačke sa slike 16 prikazan je na slici 17.



Slika 17: Konstrukcija prostog mnogougla.

Opisani metod može se usavršiti na dva načina. Ugao  $\varphi$  koji prava  $y = mx + b$  zaklapa sa  $x$  osom dobija se korišćenjem veze  $m = \tan \varphi$ , odnosno iz jednačine  $\varphi = \arctan m$ . Međutim, uglovi se ne moraju eksplicitno izračunavati. Uglovi se koriste samo za nalaženje redosleda kojim treba povezati tačke. Isti redosled dobija se uređenjem nagiba odgovarajućih polupravih; to čini nepotrebnim izračunavanje arkustangensa. Iz istog razloga nepotrebno je izračunavanje rastojanja kad dve tačke imaju isti nagib — dovoljno je izračunati kvadrate rastojanja. Dakle, nema potrebe za izračunavanjem kvadratnih korenova.

```
// ekstremna tacka
Tacka P0;

// funkcija koja menja vrednosti dvema tackama
void swap(Tacka &A, Tacka &B){

    Tacka pom = A;
    A = B;
    B = pom;
}

// racunanje kvadrata rastojanja izmedju tacaka A i B
int kvadratRastojanja(Tacka A, Tacka B){
    return (A.x - B.x)*(A.x - B.x) +
           (A.y - B.y)*(A.y - B.y);
}

// funkcija koja utvrdjuje orijentaciju tacaka P1, P2 i P3
orij orijentacija(Tacka P1, Tacka P2, Tacka P3){
    int d = (P2.x-P1.x)*(P3.y-P1.y) - (P3.x-P1.x)*(P2.y-P1.y);
    if (d == 0) return KOLINEARNE;
    else if (d > 0)
        return POZITIVNA_ORJ;
```

```

    else return NEGATIVNA_ORJ;
}

// odredjujemo i stampamo prosti mnogougao
// sa datim skupom temena
void odstampajProstMnogougao(vector<Tacka> & tacke){

    int n = tacke.size();
    // trazimo tacku sa maksimalnom x koordinatom,
    // u slucaju da ima vise tacaka sa maksimalnom x koordinatom
    // biramo onu sa najmanjom y koordinatom
    int xmin = tacke[0].x;
    int min = 0;
    for (int i = 1; i < n; i++){
        int x = tacke[i].x;
        if ((x < xmin) ||
            (xmin == x && tacke[i].y < tacke[min].y)){
            xmin = tacke[i].x;
            min = i;
        }
    }

    // ovako odredjenu tacku stavljamo na prvu poziciju
    swap(tacke[0], tacke[min]);

    P0 = tacke[0];
    // sortiramo preostalih n-1 tacaka u odnosu na prvu tacku;
    // tacka A je pre tacke B u sortiranom nizu
    // ako tacka B zahvata veci ugao nego A
    // (racunato u smeru suprotnom od smeru kazaljke)
    sort(begin(tacke)+1, end(tacke),
        [](auto& P1, auto& P2){
            // ako su kolinearne tacke
            if (orijentacija(P0,P1,P2) == KOLINEARNE) {
                // manja je ona tacka koja je bliza tacki P0
                if (kvadratRastojanja(P0,P1) <= kvadratRastojanja(P0,P2))
                    return true;
                else return false;
            }
            else if (orijentacija(P0,P1,P2) == POZITIVNA_ORJ)
                return true;
            else return false;
        });

    // stampamo skup tacaka u novom (sortiranom) poretku
    for (int i = 0; i < n; i++)

```

```

        cout << "(" << tacke[i].x << ", " << tacke[i].y <<"), ";
    }

    int main(){
        vector<Tacka> tacke = {{0, 3}, {1, 1}, {2, 2}, {4, 4},
                               {0, 0}, {1, 2}, {3, 1}, {3, 3}};
        odstampajProstMnogougao(tacke);
        return 0;
    }

```

Osnovna komponenta vremenske složenosti ovog algoritma potiče od sortiranja. Složenost algoritma je dakle  $O(n \log n)$ .